

RecoverAI — Full Project Architecture & Technical Blueprint

1. Purpose of This Document

This document defines the intended technical architecture of RecoverAI.

Claude Code must use this document as the **architectural source of truth** while implementing individual tasks.

The architecture is a target architecture.

Not every component described here has been implemented yet.

Critical rule

Do NOT assume that a component exists simply because it appears in this document.

Only implement components when the project manager explicitly requests them.

The project should be built incrementally.

2. Product Definition

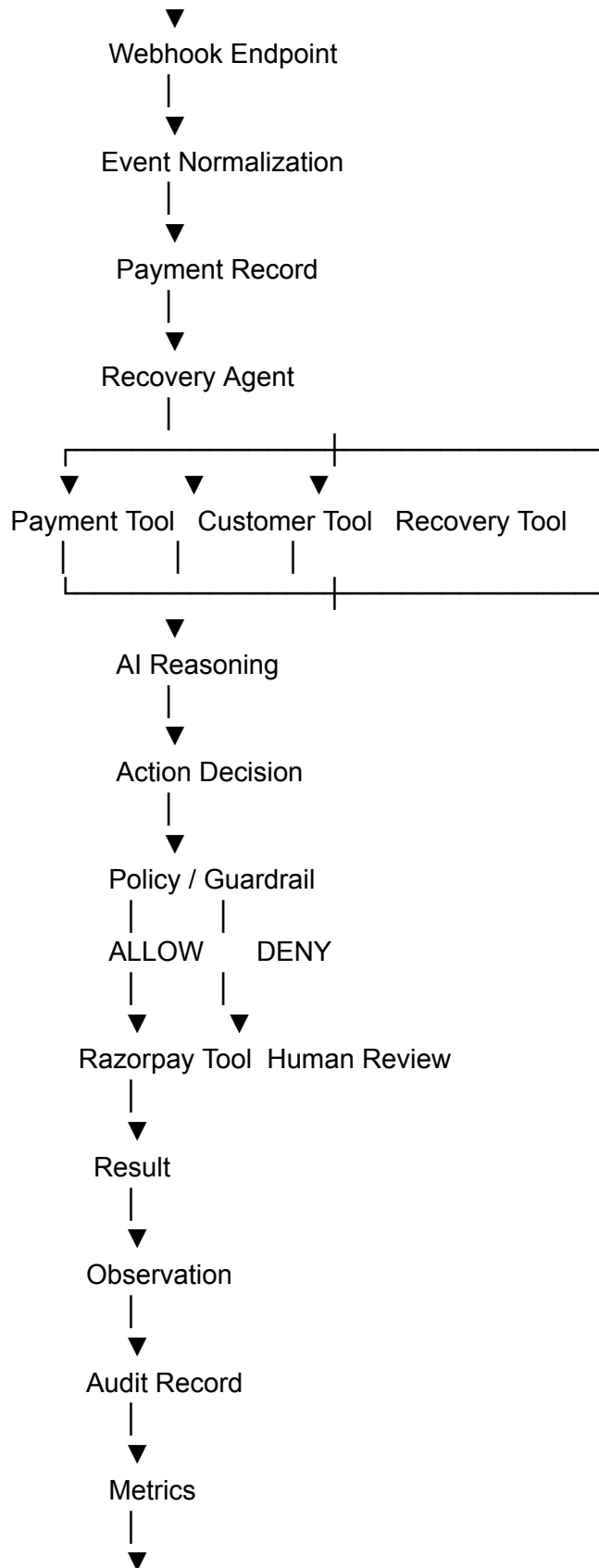
RecoverAI

RecoverAI is an AI-powered revenue recovery agent for merchants.

Its purpose is to identify payments that are at risk of being lost, investigate the payment and customer context, decide whether recovery is worthwhile, select an appropriate recovery action, execute that action through controlled backend tools, and measure the result.

The core system should eventually behave like:

```
RAZORPAY
|
| Payment Event
```



3. Core Engineering Principle

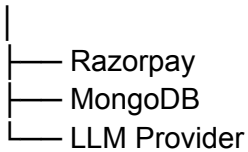
RecoverAI should be a **modular monolith**, not a collection of microservices.

For the MVP:

Frontend



Node.js / Express Backend



Do NOT create:

- Separate agent servers
- Separate tool servers
- Separate webhook services
- Separate analytics services
- Separate policy services
- Message queues

unless there is a demonstrated need later.

A modular monolith is easier to:

- build
 - debug
 - deploy
 - explain
 - demonstrate
 - maintain within the 40-hour constraint
-

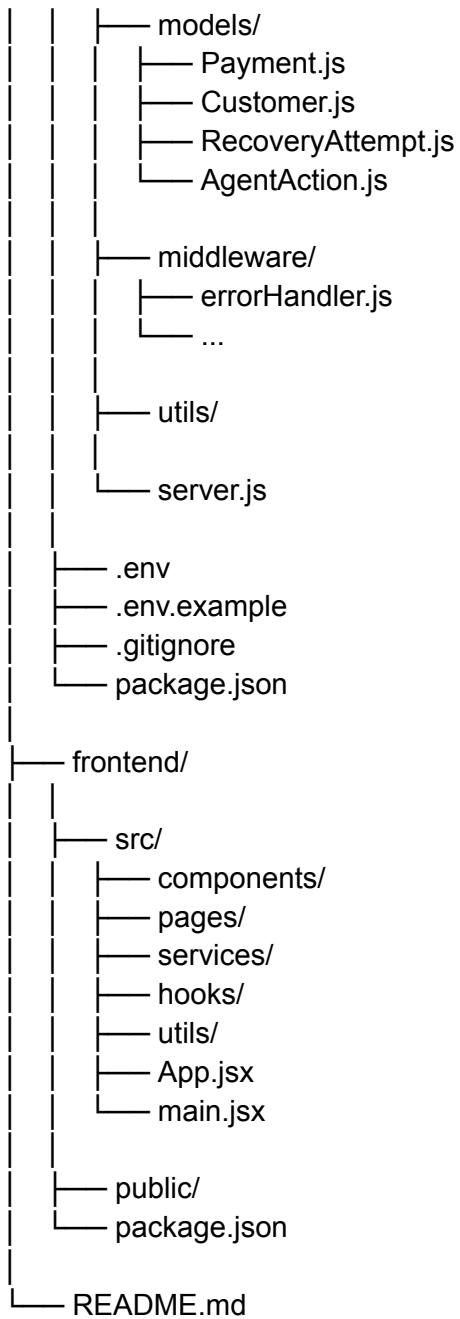
4. Repository Architecture

Current repository:

```
recover-ai/
├── backend/
└── frontend/
```

Eventually, the intended structure should evolve toward something approximately like:

```
recover-ai/
├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   └── env.js
│   │   ├── routes/
│   │   │   ├── health.routes.js
│   │   │   ├── payment.routes.js
│   │   │   ├── webhook.routes.js
│   │   │   ├── recovery.routes.js
│   │   │   └── agent.routes.js
│   │   ├── controllers/
│   │   ├── services/
│   │   │   ├── razorpay.service.js
│   │   │   ├── payment.service.js
│   │   │   ├── customer.service.js
│   │   │   ├── recovery.service.js
│   │   │   └── agent.service.js
│   │   └── agent/
│   │       ├── agent.js
│   │       ├── state.js
│   │       ├── tools/
│   │       └── policies/
│   └── ...
└── frontend/
```



Important

This is a target structure.

Do not create all these directories now.

Only create a directory when the corresponding feature is being implemented.

5. Frontend Architecture

Technology

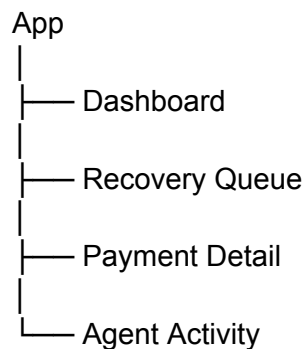
React
Vite
JavaScript
CSS

No UI library is required for the MVP.

The frontend should be a dashboard-oriented application.

6. Frontend Page Structure

Eventually:



The exact routing strategy can remain simple.

Do not introduce a routing library unless the application actually needs multiple routes.

If a single-page dashboard can handle the MVP cleanly, that is acceptable.

7. Frontend Dashboard

The dashboard should eventually communicate four things immediately:

1. Revenue at risk

Example:

₹87,400

At Risk

2. Revenue recovered

₹31,600

Recovered

3. Recovery rate

36.1%

Recovery Rate

4. Recovery queue

Example:

Customer	Amount	Reason	Action
Rahul	₹2,499	Bank error	Recover
Priya	₹899	Abandoned	Recover
Aman	₹4,999	Timeout	Review
Rohit	₹7,999	Declined	Stop

The UI should prioritize operational clarity over decoration.

8. Frontend Payment Detail

Eventually, clicking a recovery item should expose payment context.

Example:

₹2,499 AT RISK

Customer:
Rahul Sharma

Payment:
Failed

Reason:
Temporary bank failure

Previous successful payments:
3

AI Recommendation:
CREATE_PAYMENT_LINK

Confidence:
82%

Why:
Customer has historically completed payments successfully
and this appears to be a temporary failure.

[EXECUTE RECOVERY]

This screen should eventually connect to real backend data.

9. Frontend Agent Activity

The dashboard should eventually expose agent reasoning/activity.

Example:

RECOVERY AGENT

- ✓ Payment failure detected
- ✓ Customer history retrieved
- ✓ Failure investigated
- ✓ Recovery action selected
- ✓ Guardrail check passed

- ✓ Payment Link created
- ✓ Waiting for payment

This is important for the demo because it makes the agent behavior visible.

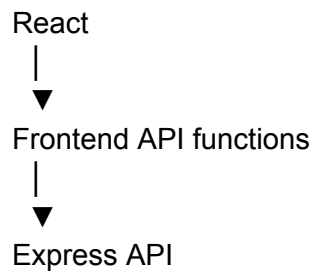
Do not fake activity merely for animation.

Whenever practical, display activity generated from actual backend state/audit records.

10. Frontend API Layer

Frontend should communicate with the backend through a small API layer.

Conceptually:



Example future functions:

`getDashboardStats()`

`getRecoveryQueue()`

`getPaymentDetails(paymentId)`

`getAgentActivity(paymentId)`

`executeRecovery(paymentId)`

Do not scatter raw `fetch()` calls throughout many components.

However, do not create an elaborate API client framework.

11. Backend Architecture

Backend technology:

Node.js
Express

The backend is the primary control layer.

It is responsible for:

- API endpoints
 - Razorpay communication
 - webhook verification
 - data persistence
 - agent execution
 - tool execution
 - guardrails
 - recovery actions
 - audit logging
-

12. Backend Layering

Use a simple layered structure:

Route

↓

Controller

↓

Service

↓

External API / Database

For agent actions:

Agent

↓
Tool
↓
Policy
↓
Service
↓
External API

Do not allow the frontend to bypass these layers.

13. API Route Groups

Eventually, the backend may expose:

GET /api/health

GET /api/dashboard

GET /api/payments/:id

GET /api/recovery

POST /api/recovery/:id/execute

POST /api/agent/run

POST /api/webhooks/razorpay

These are target endpoints.

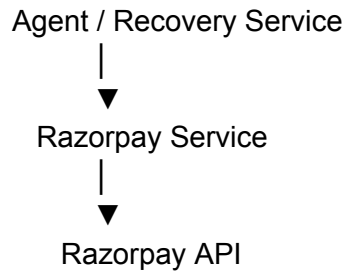
Do not create all of them at once.

Only create an endpoint when its functionality is being implemented.

14. Razorpay Integration Layer

Razorpay communication must be isolated from business logic.

Conceptually:



The Razorpay service should contain reusable functions such as:

```
createOrder()  
getPayment()  
createPaymentLink()  
fetchPaymentLink()
```

Exact functions depend on the recovery strategy selected later.

15. Razorpay Credentials

Backend environment variables may include:

```
RAZORPAY_KEY_ID  
RAZORPAY_KEY_SECRET  
RAZORPAY_WEBHOOK_SECRET
```

Potentially:

```
LLM_API_KEY  
MONGODB_URI  
PORT
```

Never expose server secrets to the frontend.

16. Existing Razorpay Functionality

The following functionality already exists:

Razorpay authentication

Tested successfully.

Order service

A reusable order creation function exists.

Conceptually:

```
createOrder({  
  amount,  
  currency,  
  receipt  
})
```

Amount is handled in paise for Razorpay.

Example:

₹2499

↓

249900 paise

Order endpoint

Existing:

POST /api/payments/orders

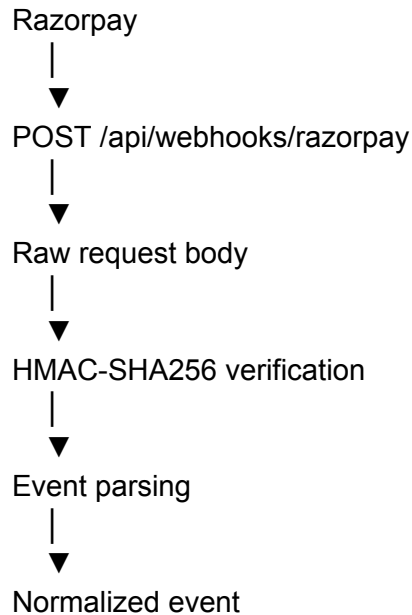
Webhook endpoint

Existing:

POST /api/webhooks/razorpay

17. Existing Webhook Architecture

Current flow:



Currently supported event types include:

payment.failed
payment.captured

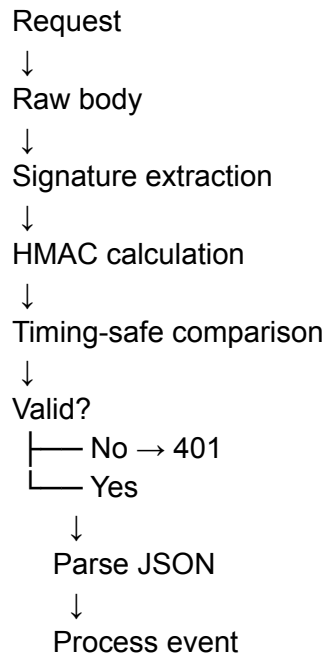
Normalized events should look approximately like:

```
{
  "event": "payment.failed",
  "paymentId": "pay_TESTFAILED001",
  "orderId": "order_TESTFAILED001",
  "amount": 249900,
  "currency": "INR",
  "status": "failed",
  "method": "card",
  "failureReason": "Payment failed due to insufficient funds"
}
```

18. Webhook Security

Signature verification must happen before trusting webhook data.

Conceptually:

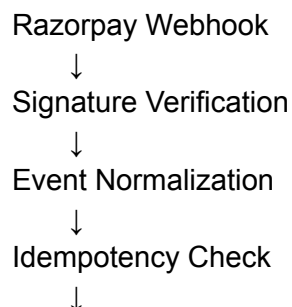


Do not change raw-body handling casually.

19. Webhook Persistence

This is a future component.

Eventually:



MongoDB
↓
Agent Trigger

Do not assume this is currently implemented.

20. Database Architecture

MongoDB / MongoDB Atlas will eventually store the application state.

The database exists primarily to provide:

- persistent payment state
 - customer history
 - recovery history
 - agent state
 - auditability
 - metrics
-

21. Payment Model

Conceptual payment document:

```
{  
  razorpayPaymentId,  
  razorpayOrderId,  
  amount,  
  currency,  
  status,  
  method,  
  failureReason,  
  customerId,  
  createdAt,  
  updatedAt  
}
```


Potential statuses:

created
failed
captured
recovered

Exact statuses should be finalized when implementation begins.

Do not create unnecessary fields.

22. Customer Model

Conceptual customer document:

```
{  
  customerId,  
  name,  
  email,  
  phone,  
  successfulPayments,  
  failedPayments,  
  totalSuccessfulAmount,  
  recoveryAttempts,  
  optedOut,  
  createdAt,  
  updatedAt  
}
```

This allows the agent to reason about customer history.

Example:

Successful payments: 4
Failed payments: 1
Previous recovery attempts: 0
Opted out: false

23. Recovery Attempt Model

Conceptual document:

```
{  
  paymentId,  
  customerId,  
  action,  
  status,  
  amount,  
  reason,  
  agentDecision,  
  confidence,  
  guardrailResult,  
  externalReference,  
  createdAt,  
  updatedAt  
}
```

Potential statuses:

pending
allowed
denied
executed
failed
succeeded
human_review

Do not implement all statuses unless needed.

24. Audit Model

The audit trail should eventually answer:

What happened?

Why did the agent make this decision?

Which tools were called?

What did the tools return?

Did the policy allow the action?

What happened afterward?

Conceptual audit record:

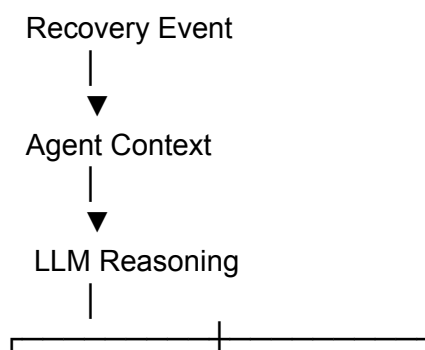
```
{  
  paymentId,  
  event,  
  actor,  
  action,  
  reason,  
  tool,  
  toolInput,  
  toolOutput,  
  policyDecision,  
  timestamp  
}
```

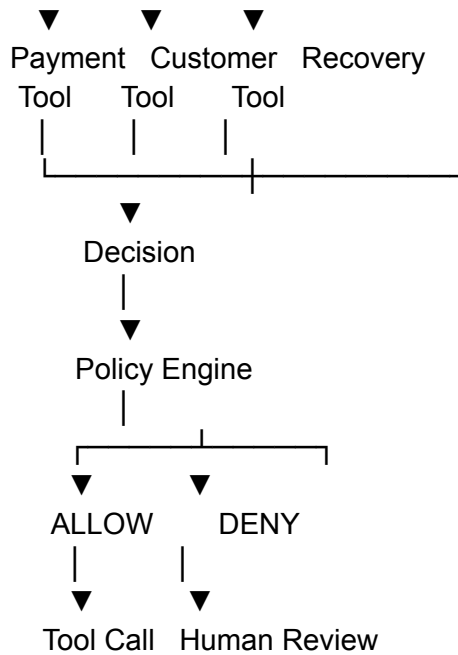
Do not store sensitive information unnecessarily.

25. Agent Architecture

The agent is the core intelligence layer.

Conceptually:





26. Agent State

The agent should eventually operate using a structured state.

Conceptually:

```
{  
  payment,  
  customer,  
  recoveryHistory,  
  availableActions,  
  previousToolResults,  
  decision,  
  policyResult,  
  outcome  
}
```

Do not introduce a complicated state-management framework.

A plain JavaScript object may be sufficient.

27. Agent Input

The agent should eventually receive normalized information rather than arbitrary raw database documents.

Example:

```
{
  "payment": {
    "id": "pay_123",
    "amount": 2499,
    "status": "failed",
    "failureReason": "Temporary bank failure"
  },
  "customer": {
    "successfulPayments": 4,
    "failedPayments": 1
  },
  "recovery": {
    "previousAttempts": 0
  }
}
```

28. Agent Tools

The agent should eventually have a small set of narrowly scoped tools.

Tool 1

get_payment_details(paymentId)

Purpose:

Retrieve payment information.

Tool 2

get_customer_history(customerId)

Purpose:

Retrieve relevant historical payment behavior.

Tool 3

get_recovery_history(paymentId)

Purpose:

Determine whether recovery has already been attempted.

Tool 4

create_payment_link(...)

Purpose:

Create a controlled Razorpay payment link.

This tool must NOT bypass policy checks.

Tool 5

check_payment_status(paymentId)

Purpose:

Determine whether the payment has already succeeded or remains unresolved.

Tool 6

`record_recovery_action(...)`

Purpose:

Persist the agent's action and outcome.

Tool 7

`escalate_to_human(...)`

Purpose:

Move cases outside the automatic recovery boundary to human review.

29. Tool Design Principle

Each tool should have:

Clear input



Validation



Single responsibility



Clear output

Avoid tools that perform multiple unrelated operations.

Bad:

`recoverCustomerAndCreateLinkAndUpdateDatabase()`

Better:

`get_customer_history()`

`create_payment_link()`

record_recovery_action()

30. LLM Architecture

The LLM should be responsible for:

- interpreting context
- reasoning about failure
- selecting an action
- explaining the decision
- requesting tools

The LLM should NOT be responsible for:

- authenticating with Razorpay
 - deciding whether an action violates policy
 - directly accessing MongoDB
 - directly calling arbitrary HTTP endpoints
 - managing secrets
-

31. LLM Tool-Calling Flow

Eventually:

Agent

↓

LLM

↓

LLM requests tool

↓

Backend validates tool request

↓

Tool executes

↓

Tool result returned to LLM

↓

LLM continues reasoning

↓
Final structured decision

Example:

LLM:

"Call get_customer_history(customer_123)"

Backend:

→ validates request

Tool:

→ retrieves customer history

Result:

→ 4 successful payments

→ 1 failed payment

→ 0 recovery attempts

LLM:

→ recommends CREATE_PAYMENT_LINK

32. Agent Decision Schema

A future decision may resemble:

```
{  
  "action": "CREATE_PAYMENT_LINK",  
  "confidence": 0.82,  
  "reason": "Customer has a strong successful payment history and this appears to be a  
temporary failure.",  
  "requiresHumanReview": false  
}
```

Potential actions:

CREATE_PAYMENT_LINK

RETRY

NO_ACTION

STOP
HUMAN_REVIEW

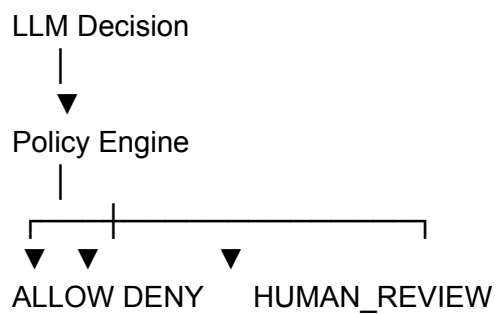
The exact action set should be finalized during implementation.

Do not invent additional actions without need.

33. Policy Engine

The policy engine is separate from the LLM.

Architecture:



The policy engine should be deterministic.

The LLM can recommend.

The policy engine decides whether that recommendation is permitted.

34. Example Guardrails

Potential rules:

IF payment already succeeded
→ STOP

IF recovery attempts $\geq$ configured limit
→ STOP

IF amount > automatic recovery limit
→ HUMAN_REVIEW

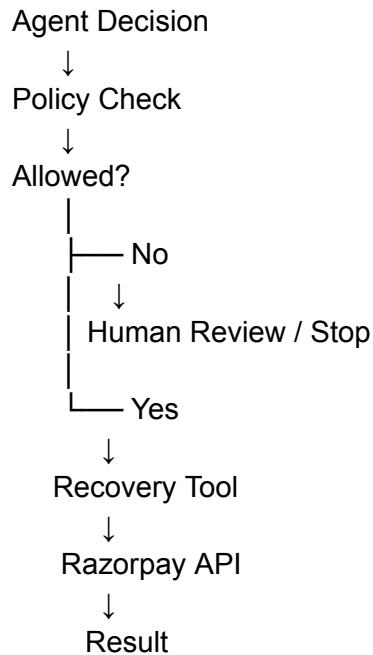
IF customer opted out
→ STOP

Exact values will be explicitly configured later.

Never invent financial thresholds.

35. Recovery Execution

Eventually:



The frontend must not directly execute privileged Razorpay operations.

36. Recovery Strategy

The MVP should focus on a small number of realistic recovery actions.

Primary candidate:

CREATE_PAYMENT_LINK

This is preferable to building many recovery mechanisms.

The system can later add additional actions if time permits.

37. Recovery Outcome

After an action:

Recovery Action



Check Result



Record Outcome



Update Payment



Update Recovery Attempt



Update Metrics

Possible outcome:

SUCCESS

FAILED

PENDING

EXPIRED

Exact implementation depends on the Razorpay recovery mechanism selected.

38. Idempotency

Webhook processing and recovery actions must eventually avoid duplicate execution.

Example:

Same Razorpay webhook arrives twice



Detect existing event



Do not process twice

Likewise:

Recovery action already executed



Do not create another recovery action automatically

Idempotency is a reliability feature and should be implemented before production-like deployment.

39. Metrics Architecture

RecoverAI should eventually calculate:

Revenue at risk

Sum of eligible failed/unrecovered payment amounts.

Revenue At Risk =

Σ eligible unrecovered payments

Revenue recovered

Sum of successfully recovered amounts.

Revenue Recovered =

Σ successfully recovered payments

Recovery attempts

Number of recovery actions attempted

Successful recoveries

Number of recovery actions that succeeded

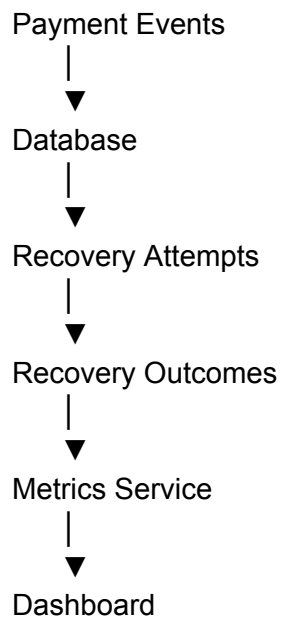
Recovery rate

Conceptually:

Recovery Rate =
Recovered Revenue / Revenue At Risk × 100

The exact business definition may be refined later.

40. Metrics Flow



Do not calculate important metrics from hardcoded frontend values once backend data exists.

41. Audit Trail Architecture

Every important agent decision should eventually produce an auditable record.

Example:

10:31:02

Payment failure detected

10:31:03

Payment details retrieved

10:31:03

Customer history retrieved

10:31:04

Recovery history retrieved

10:31:05

AI recommendation:

CREATE_PAYMENT_LINK

10:31:05

Guardrail:

ALLOWED

10:31:06

Payment Link created

10:31:06

Waiting for payment

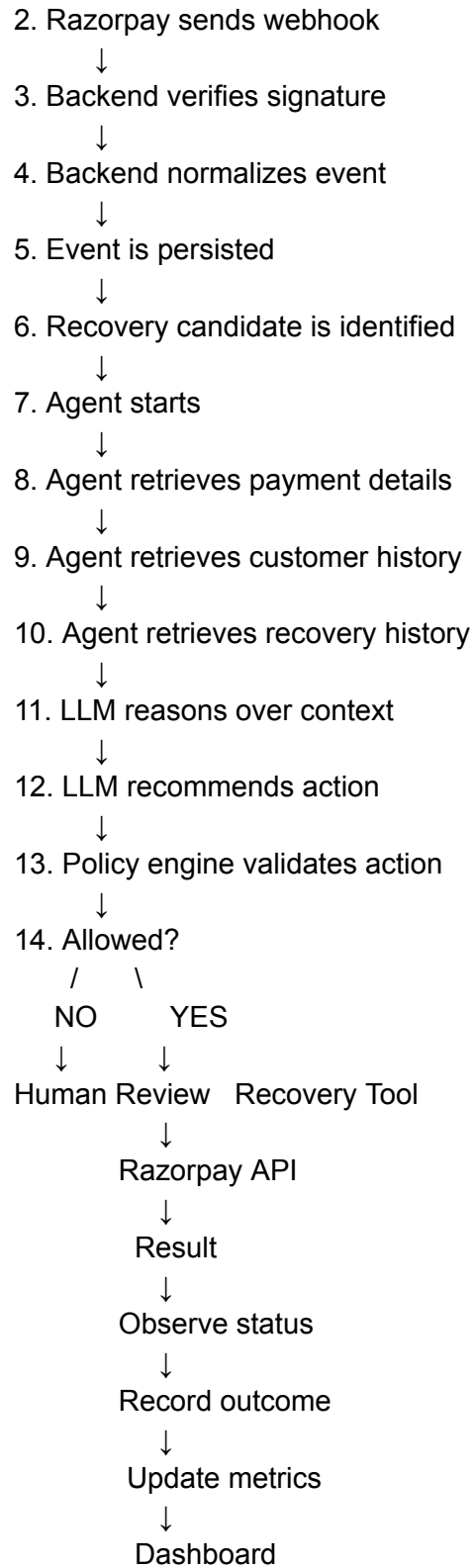
This should eventually be backed by real state rather than merely frontend animation.

42. End-to-End Agent Flow

The intended final flow is:

1. Razorpay payment fails

↓



This is the core architecture of RecoverAI.

43. Example End-to-End Scenario

Suppose:

Customer: Rahul Sharma

Payment:

₹2,499

Status:

Failed

Reason:

Temporary bank failure

Previous successful payments:

4

Previous failed payments:

0

Previous recovery attempts:

0

The agent receives the event.

Step 1 — Investigate

`get_payment_details()`

Returns:

₹2,499

Failed

Temporary bank failure

Step 2 — Customer investigation

`get_customer_history()`

Returns:

4 successful payments
0 previous failures

Step 3 — Recovery history

get_recovery_history()

Returns:

0 previous attempts

Step 4 — Reason

LLM concludes:

Customer appears likely to complete payment.

Recommended action:
CREATE_PAYMENT_LINK

Step 5 — Guardrail

Policy checks:

Payment already succeeded? NO

Recovery attempts exceeded? NO

Amount above automatic limit? NO

Customer opted out? NO

Result:

ALLOW

Step 6 — Execute

Backend calls:

`create_payment_link()`

Step 7 — Observe

System checks payment status.

Step 8 — Record

If successful:

Recovered revenue += ₹2,499

Step 9 — Dashboard

The dashboard updates:

Revenue Recovered
₹31,600 → ₹34,099

44. Human Review Flow

Not every payment should be automatically recovered.

Example:

Payment:
₹50,000

Agent recommendation:
`CREATE_PAYMENT_LINK`

Policy:

Automatic recovery limit:
₹10,000

Result:

DENY AUTOMATIC EXECUTION

Then:

HUMAN_REVIEW

The agent does not get to bypass this.

This is important to demonstrate responsible agent design.

45. Failure Handling

External systems can fail.

Example:

Agent

↓

create_payment_link()

↓

Razorpay API failure

↓

Tool returns structured error

↓

Agent observes failure

↓

Recovery attempt recorded as failed

↓

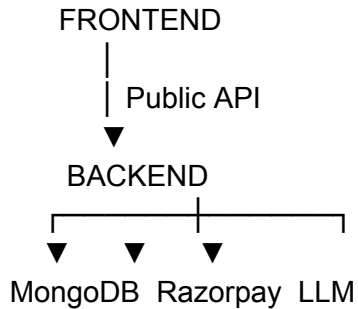
No uncontrolled retry

Retries should only be introduced where they are actually safe.

Never blindly retry financial operations.

46. Security Architecture

Security boundaries:



Secrets exist only on the backend.

The frontend receives sanitized data.

Never send:

RAZORPAY_KEY_SECRET
RAZORPAY_WEBHOOK_SECRET
LLM_API_KEY
MONGODB_URI

to the browser.

47. Configuration Architecture

Environment configuration belongs in backend environment variables.

Example:

PORT=
MONGODB_URI=

RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=

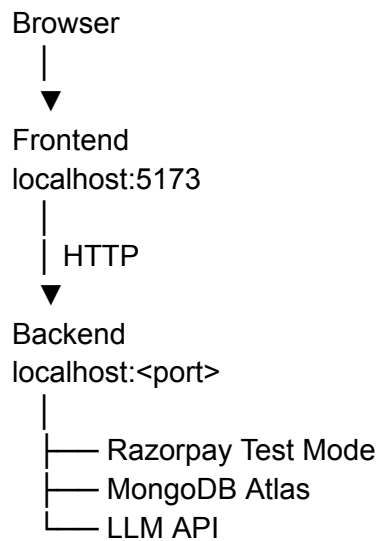
LLM_API_KEY=

Actual variable names may change only through an explicit implementation decision.

Never hard-code secrets.

48. Local Development Architecture

Development environment:



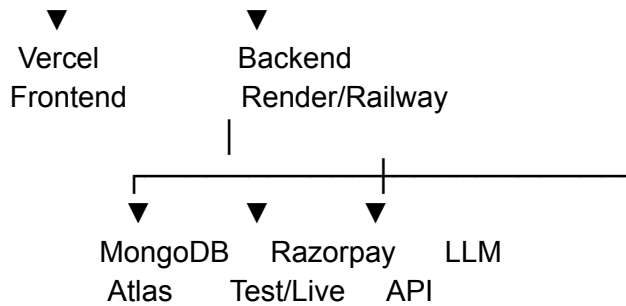
Razorpay webhook testing may initially be performed using signed local test requests.

Later, an actual Razorpay Dashboard webhook can be connected.

49. Deployment Architecture

Target deployment:





Exact backend deployment provider is not finalized.

Do not deploy until explicitly instructed.

50. Environment Separation

Development:

Razorpay Test Mode
Development MongoDB
Development LLM key

Production/demo deployment:

Configured environment variables
Razorpay environment selected deliberately
Production/demo database

Never accidentally mix credentials or databases.

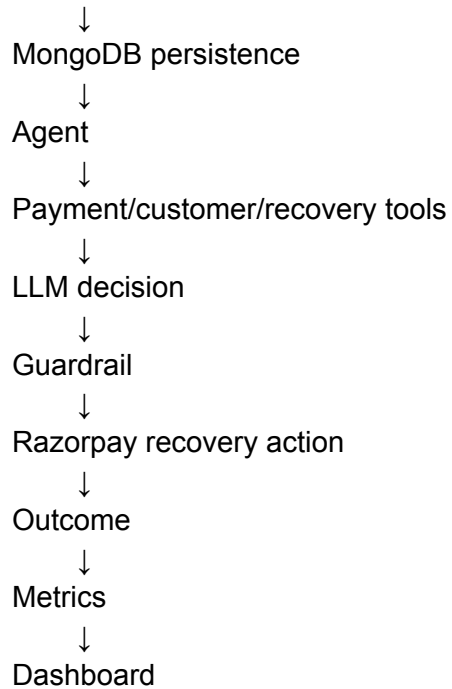
51. MVP Scope

The MVP should prioritize this path:

Razorpay failed payment



Webhook



Everything outside this path is secondary.

52. MVP Priority Order

When time is limited, prioritize:

P0 — Must Work

Backend

- ✓ Razorpay integration
- ✓ Webhook
- ✓ Payment persistence
- ✓ Agent
- ✓ At least 2–3 useful tools
- ✓ Guardrails
- ✓ One real recovery action
- ✓ Recovery outcome
- ✓ Basic metrics
- ✓ Basic dashboard

P1 — Strong Demo Improvements

- ✓ Agent activity
- ✓ Audit trail
- ✓ Better payment detail
- ✓ Better recovery queue
- ✓ Error handling
- ✓ Idempotency

P2 — Optional

- More recovery strategies
- Better analytics
- More sophisticated confidence scoring
- Recovery probability model
- Advanced visualizations
- Additional automation

P2 features must never delay the core end-to-end system.

53. What NOT To Build

Unless explicitly requested, do not build:

- ✗ Microservices
- ✗ Kubernetes
- ✗ Redis
- ✗ Kafka
- ✗ Complex message queues
- ✗ Multi-agent architecture
- ✗ Custom ML model
- ✗ Vector database
- ✗ RAG system
- ✗ Complex authentication
- ✗ Role-based access control
- ✗ Enterprise permissions
- ✗ Complex state-management framework
- ✗ Large UI component library
- ✗ Advanced observability platform

These are unnecessary for the MVP.

54. AI/ML Optional Enhancement

Only after the complete recovery loop works may we consider a simple recovery-probability model.

Potential input features:

customer successful payment count
customer failure count
payment amount
failure type
previous recovery success
time since previous payment

Potential output:

recovery probability

Example:

Recovery probability: 82%

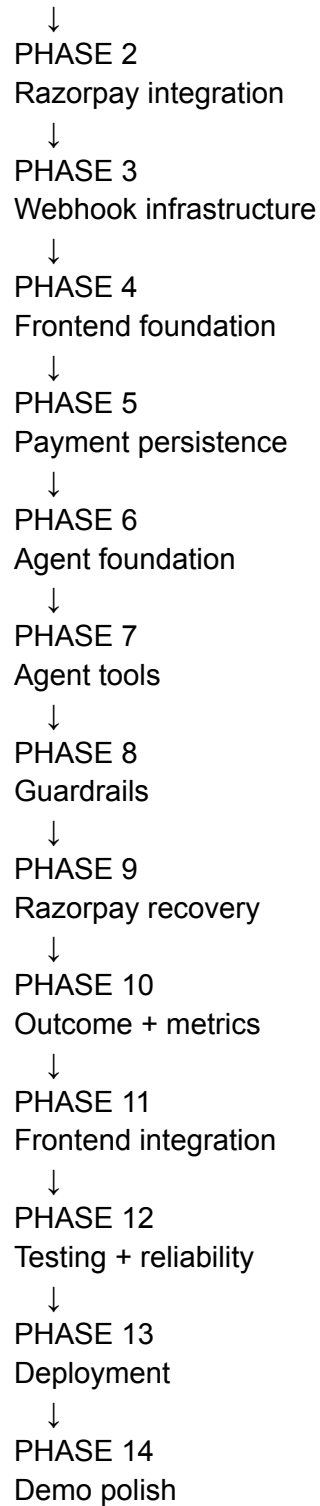
This is optional.

Do not build this before the agent and recovery loop work.

55. Architecture Evolution

The project should evolve in this order:

PHASE 1
Foundation



The project manager may intentionally move between frontend and backend rather than following these phases strictly.

56. Current Implementation Status

At the time this architecture document was created:

Backend

DONE

- ✓ Node.js
- ✓ Express
- ✓ dotenv
- ✓ CORS
- ✓ GET /api/health
- ✓ .env
- ✓ .env.example
- ✓ .gitignore

- ✓ Razorpay authentication
- ✓ Razorpay test connection

- ✓ Razorpay order service
- ✓ POST /api/payments/orders

- ✓ Razorpay webhook endpoint
- ✓ Raw body handling
- ✓ HMAC-SHA256 signature verification
- ✓ Timing-safe comparison
- ✓ payment.failed parsing
- ✓ payment.captured parsing
- ✓ Normalized payment event
- ✓ Invalid signature handling
- ✓ Malformed JSON handling
- ✓ Unsupported event handling

NOT DONE

- X MongoDB
- X Payment persistence
- X Customer persistence
- X Recovery persistence
- X Webhook idempotency
- X Actual Razorpay Dashboard webhook connection
- X Agent

- X Agent state
 - X Agent tools
 - X LLM integration
 - X Tool calling
 - X Guardrails
 - X Recovery action
 - X Recovery outcome
 - X Metrics
 - X Audit trail
-

57. Current Frontend Status

DONE

- ✓ React
- ✓ Vite
- ✓ JavaScript
- ✓ Basic RecoverAI page
- ✓ Minimal styling

NOT DONE

- X Dashboard
- X Sidebar
- X Design system
- X Metric cards
- X Recovery queue
- X Payment detail
- X Agent activity
- X Backend integration
- X Real metrics

Important

A previous large frontend design-system task was aborted.

Do NOT assume any of its work exists.

The frontend baseline is the state produced by **Frontend Prompt 1**.

Do not redo the aborted Frontend 2A task unless explicitly instructed.

58. Current Development Strategy

Development is intentionally vertical.

Instead of:

Finish entire backend
↓
Finish entire frontend
↓
Add AI

we prefer:

Backend foundation
↓
Small frontend improvement
↓
Database
↓
Small frontend integration
↓
Agent foundation
↓
Agent tools
↓
Recovery
↓
Frontend integration

This allows the project to remain demonstrable throughout development.

59. Implementation Discipline

For every prompt:

1. Inspect

Understand the current implementation.

2. Locate

Identify the smallest set of files involved.

3. Implement

Make only the requested change.

4. Verify

Run the relevant test or application.

5. Report

Briefly state what changed.

6. Stop

Wait for the next instruction.

60. Architectural Decision Rule

When choosing between two implementations, prefer the one that is:

1. Easier to understand
2. Easier to test
3. Easier to explain in an interview
4. Smaller
5. More reliable
6. Sufficient for the current requirement

Do not choose a technology because it sounds more advanced.

61. Interview Defensibility

Every major architectural decision should eventually have a simple explanation.

Examples:

Why Node/Express?

It provides a simple backend for handling Razorpay webhooks, API requests, agent orchestration, and external service calls.

Why MongoDB?

The recovery system has flexible event, customer, agent, and audit data, and MongoDB is sufficient for the MVP.

Why an LLM instead of ML?

The core problem requires contextual reasoning and tool selection rather than predictive model training.

Why guardrails outside the LLM?

The model can recommend an action, but deterministic backend policies must control financial operations.

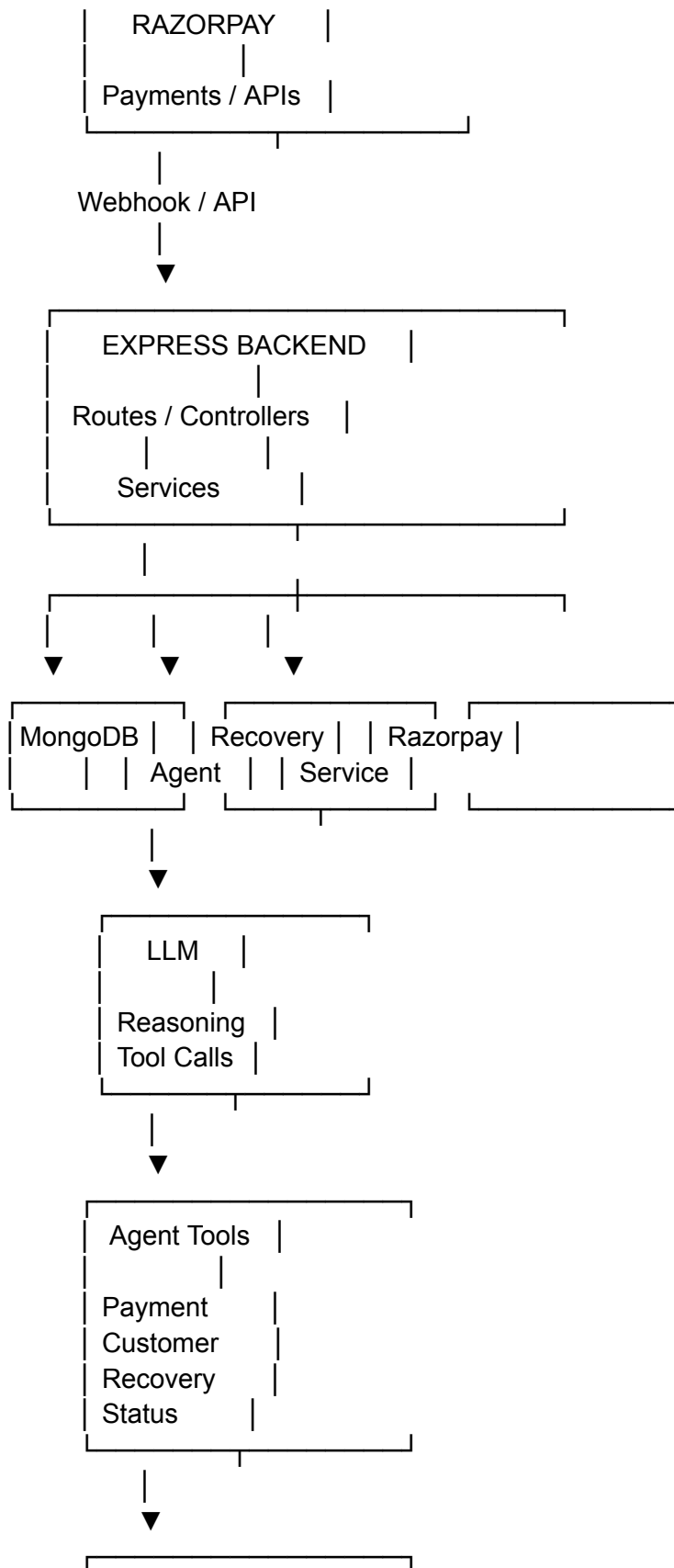
Why modular monolith?

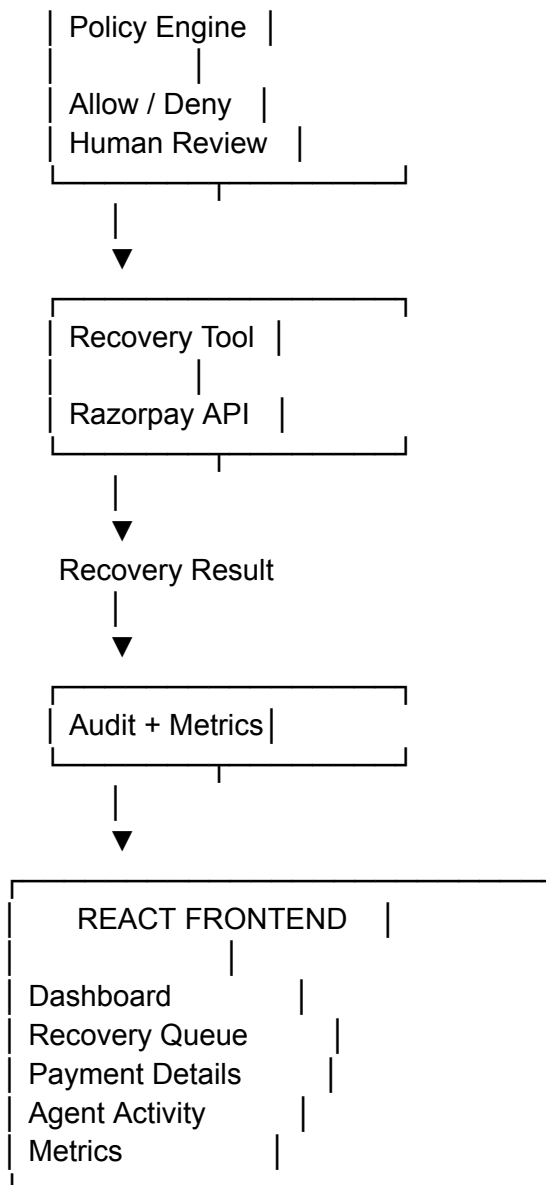
The project is small enough that microservices would add operational complexity without meaningful benefit.

These decisions should remain reflected in the implementation.

62. Final Architecture

The final conceptual architecture is:





63. One-Sentence Architecture Summary

RecoverAI is a **Node/Express modular monolith** where **Razorpay events enter through verified webhooks**, **MongoDB provides persistent payment/customer/recovery state**, an **LLM-based agent investigates and recommends recovery actions through controlled tools**, **deterministic backend guardrails control financial execution**, **Razorpay performs the recovery action**, and a **React dashboard exposes the resulting decisions, audit trail, and recovered revenue**.

64. Final Rule for Claude Code

This architecture is a **guide, not permission to build everything described here.**

For every individual implementation prompt:

Read the architecture



Understand where the requested feature belongs



Inspect the existing code



Implement **ONLY** the requested feature



Preserve the architecture



Test



STOP

Never jump ahead.

Never implement future phases.

Never assume unfinished components exist.

Never optimize for code volume.

The goal is to gradually turn this architecture into a **working, explainable, reliable revenue-recovery agent.**